

Proiectarea cu Microprocesoare
ADC și memorii ATMega324a

Andrei Voinescu

POLITEHNICA University of Bucharest

1 aprilie 2020



Outline

ADC

EEPROM

Fuse bits

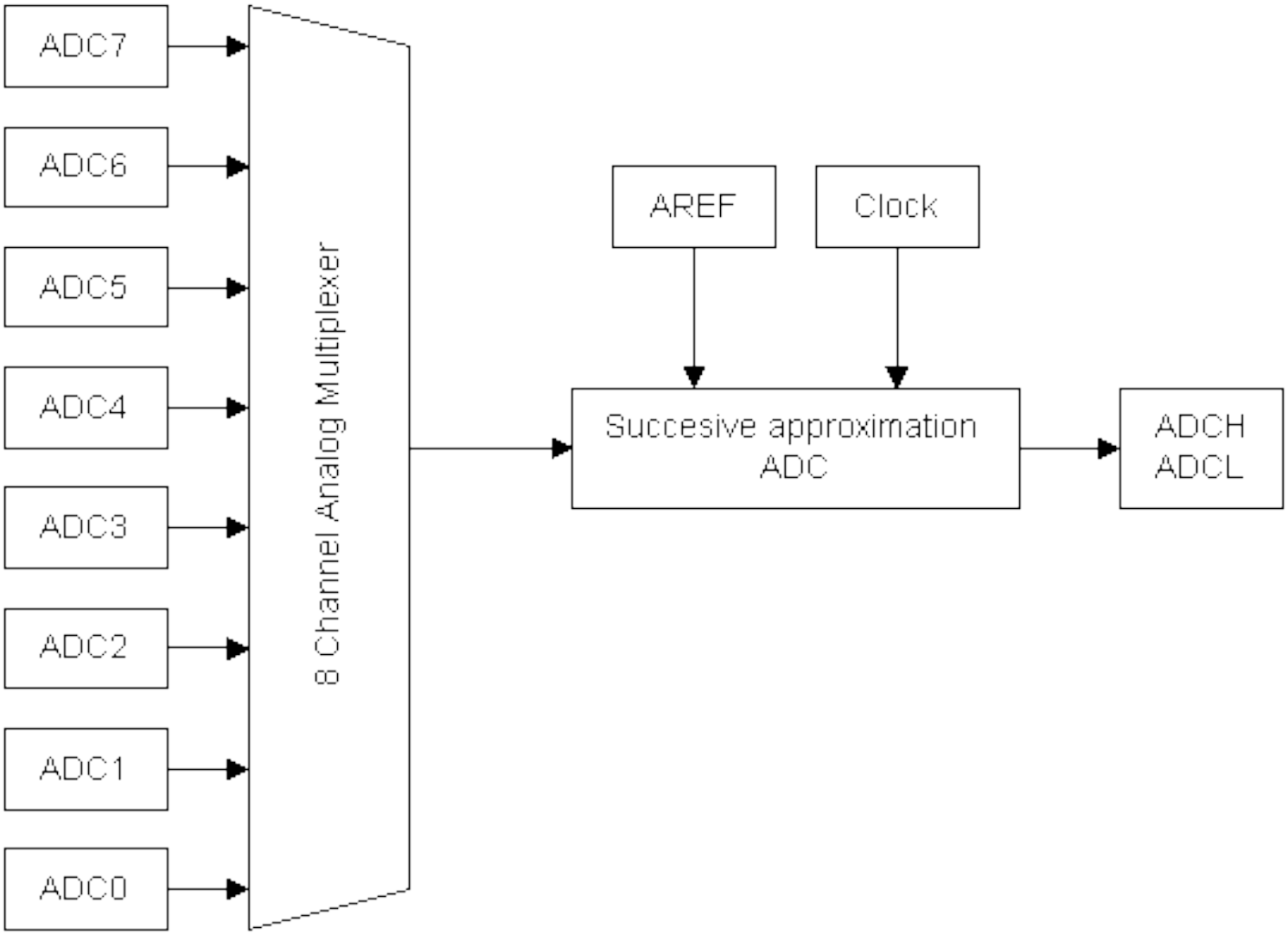
Bootloader support

- Analog to Digital Converter
  - Periferic care se ocupă de conversia unor tensiuni pe pini în valori digitale.

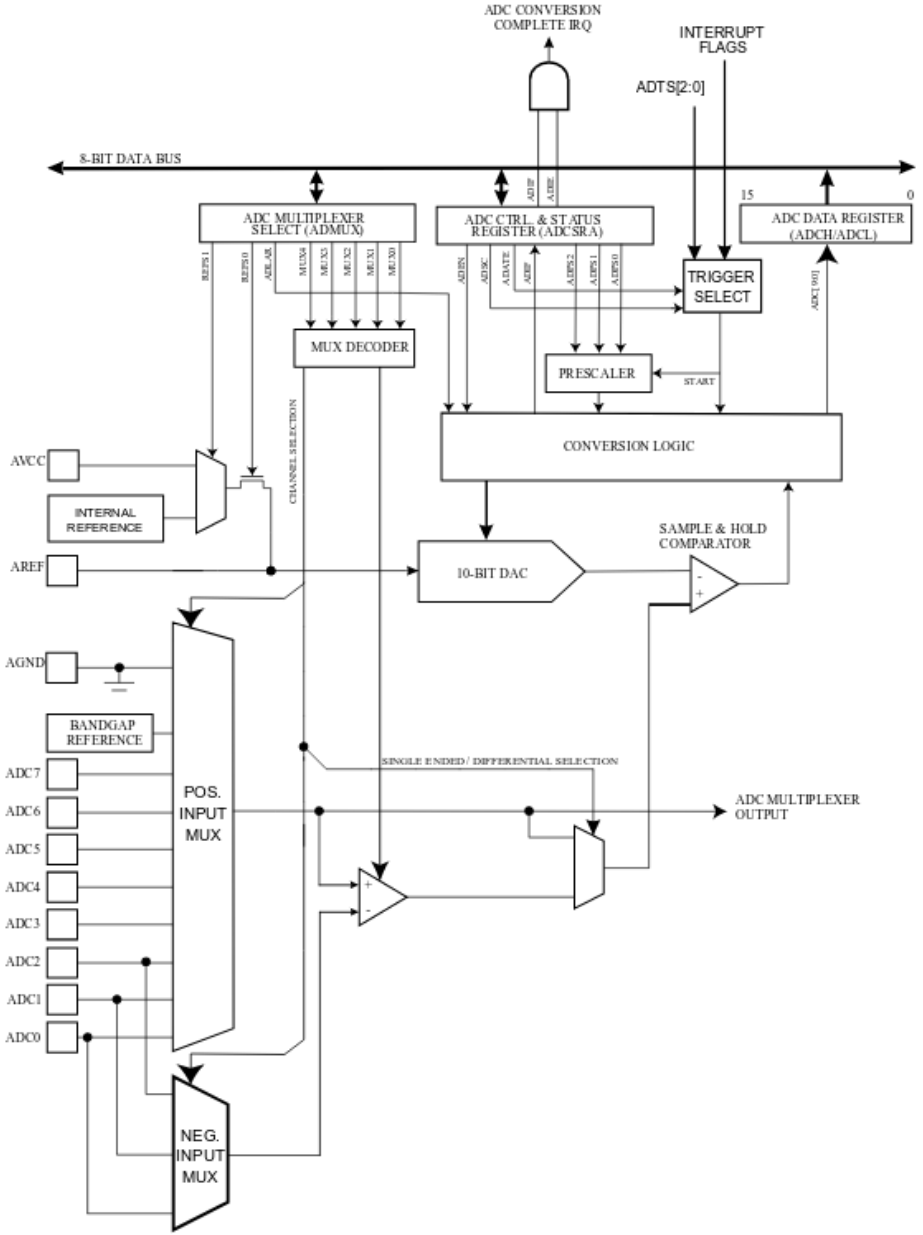
- Analog to Digital Converter
  - Periferic care se ocupă de conversia unor tensiuni pe pini în valori digitale.
  - Util pentru senzori analogici (temperatură, presiune, lumină, curent etc.)
- Rezoluție de 10 biți pe ATMega324

- Analog to Digital Converter
  - Periferic care se ocupă de conversia unor tensiuni pe pini în valori digitale.
  - Util pentru senzori analogici (temperatură, presiune, lumină, curent etc.)
- Rezoluție de 10 biți pe ATMega324
  - Rezoluție vs precizie
  - Rezoluție 10 biți - 1024 valori
  - Domeniu $[0..5V] \rightarrow 4.88mV$
  - Precizie $2LSB = 9.76mV$

ADC - funcționare



ADC - schemă bloc completă



[0] Sursă: Datasheet ATMega324a



ADC - ce trebuie să configurăm

- Cât de repede se face conversia
  - Prescaler 1..128



ADC - ce trebuie să configurăm

- Cât de repede se face conversia
  - Prescaler 1..128
  - Frecvența de lucru trebuie să fie între 50 și 200kHz



ADC - ce trebuie să configurăm

- Cât de repede se face conversia
  - Prescaler 1..128
  - Frecvența de lucru trebuie să fie între 50 și 200kHz
  - Frecvență mai mare = precizie mai mică



ADC - ce trebuie să configurăm

- Cât de repede se face conversia
  - Prescaler 1..128
  - Frecvența de lucru trebuie să fie între 50 și 200kHz
  - Frecvență mai mare = precizie mai mică
  - Scădem rezoluția = se mărește precizia



ADC - ce trebuie să configurăm

- Cât de repede se face conversia
  - Prescaler 1..128
  - Frecvența de lucru trebuie să fie între 50 și 200kHz
  - Frecvență mai mare = precizie mai mică
  - Scădem rezoluția = se mărește precizia
- Care este domeniul $[0..x]$?



ADC - ce trebuie să configurăm

- Cât de repede se face conversia
  - Prescaler 1..128
  - Frecvența de lucru trebuie să fie între 50 și 200kHz
  - Frecvență mai mare = precizie mai mică
  - Scădem rezoluția = se mărește precizia
- Care este domeniul $[0..x]$?
  - V_{cc}
  - referință externă
  - 2.56V generați intern
  - 1.1V generați intern



ADC - ce trebuie să configurăm

- Cât de repede se face conversia
  - Prescaler 1..128
  - Frecvența de lucru trebuie să fie între 50 și 200kHz
  - Frecvență mai mare = precizie mai mică
  - Scădem rezoluția = se mărește precizia
- Care este domeniul $[0..x]$?
  - V_{cc}
  - referință externă
  - 2.56V generați intern
  - 1.1V generați intern
- De unde luăm tensiunea



ADC - ce trebuie să configurăm

- Cât de repede se face conversia
  - Prescaler 1..128
  - Frecvența de lucru trebuie să fie între 50 și 200kHz
  - Frecvență mai mare = precizie mai mică
  - Scădem rezoluția = se mărește precizia
- Care este domeniul $[0..x]$?
  - V_{cc}
  - referință externă
  - 2.56V generați intern
  - 1.1V generați intern
- De unde luăm tensiunea
  - Un pin - PA0..PA7
  - Diferența între doi pini amplificată



ADC - ce trebuie să configurăm

- Cât de repede se face conversia
  - Prescaler 1..128
  - Frecvența de lucru trebuie să fie între 50 și 200kHz
  - Frecvență mai mare = precizie mai mică
  - Scădem rezoluția = se mărește precizia
- Care este domeniul $[0..x]$?
  - V_{cc}
  - referință externă
  - 2.56V generați intern
  - 1.1V generați intern
- De unde luăm tensiunea
  - Un pin - PA0..PA7
  - Diferența între doi pini amplificată
- Când facem conversia

- Cât de repede se face conversia
  - Prescaler 1..128
  - Frecvența de lucru trebuie să fie între 50 și 200kHz
  - Frecvență mai mare = precizie mai mică
  - Scădem rezoluția = se mărește precizia
- Care este domeniul $[0..x]$?
  - V_{cc}
  - referință externă
  - 2.56V generați intern
  - 1.1V generați intern
- De unde luăm tensiunea
  - Un pin - PA0..PA7
  - Diferența între doi pini amplificată
- Când facem conversia
  - La comandă
  - La un anumit semnal

- ADMUX
  - REFS - referința de tensiune
  - ADLAR - 10biți aliniați la stânga sau dreapta
  - MUX - selectarea canalului de pe care se face conversia
- ADCSRA
  - ADEN - activarea perifericului
  - ADSC - '1' pentru pornirea unei conversii
  - ADATE - activarea auto-triggering
  - ADIF - '1' când se termină o conversie
  - ADIE - activarea întreruperii de conversion complete
  - ADPS - prescaler pentru aproximațiile succesive ale rezultatului
- ADCSRB
  - ADTS - sursa de trigger
- ADC - Rezultatul conversiei

- Folosire simplă: citire de pe canalul 1

```
uint16_t val;
// initializare domeniu 0..Vcc, canalul 1
ADMUX = (1 << REFS0) | (1 << MUX0);
ADCSRA = (1 << ADEN);

// start conversie
ADCSRA |= (1 << ADSC);

// asteapta conversia sa se termine
loop_until_bit_is_set(ADCSRA, ADIF);
// valoarea este in registrul de 16 biti ADC
val = ADC;
```

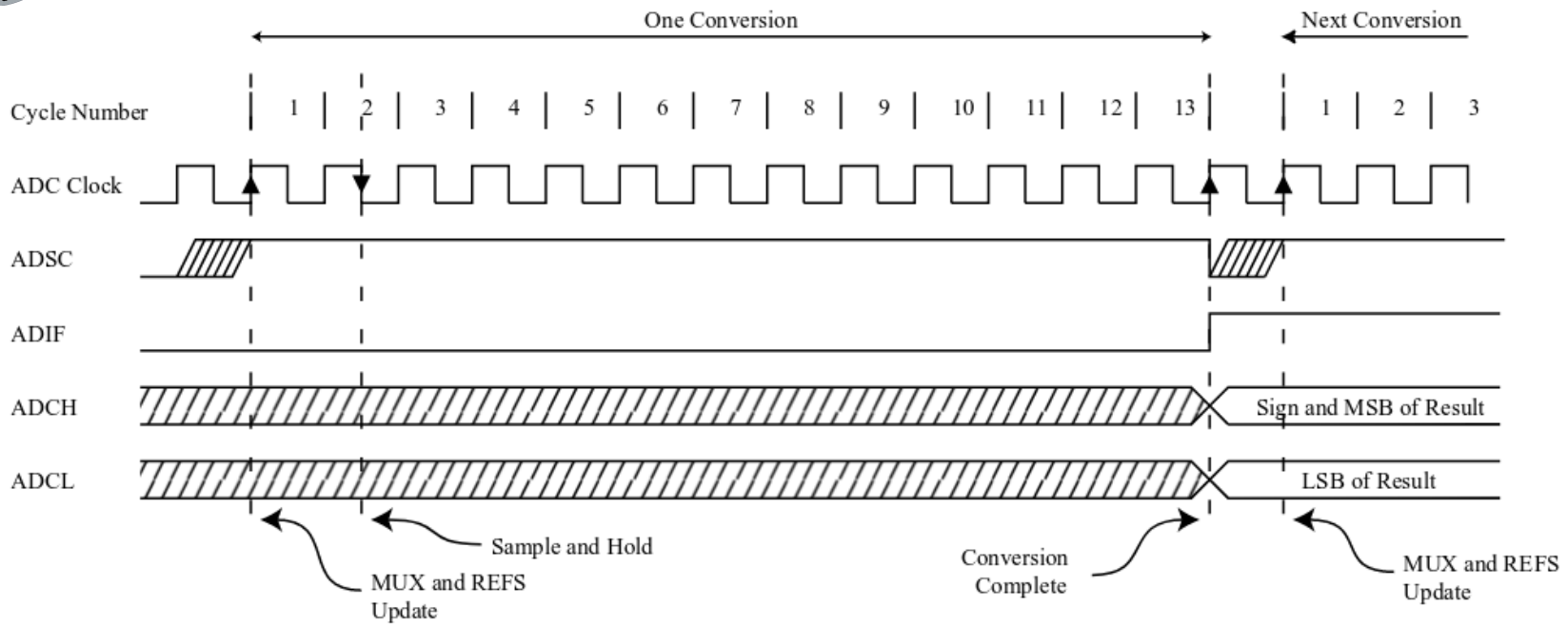
Folosire cu intreruperi - citire de pe canalul 1

```
volatile int16_t val = -1;
ISR(ADC_vect)
{
  val = ADC;
}
void init_adc()
{
  // initializare domeniu 0..Vcc, canalul 1
  ADMUX = (1 << REFS0) | (1 << MUX0);
  ADCSRA = (1 << ADEN);
}
void main()
{
  init_adc();
  sei();

  while(1)
  {
    // start conversie
    ADCSRA |= (1 << ADSC);

    if (val != -1)
    {
      // use val
    }
    _delay_ms(500);
  }
}
```

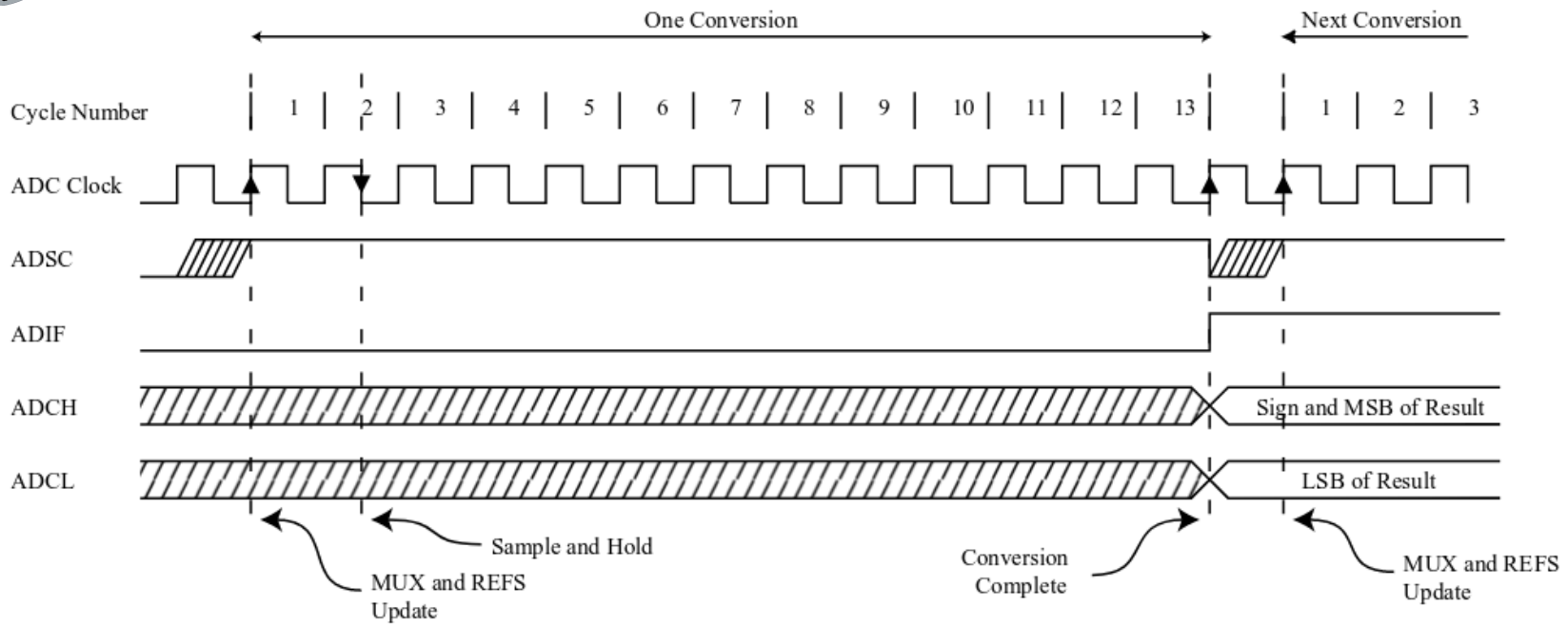
ADC - timing pentru o conversie simplă



- O conversie durează 13.x cicli ADC (prima 25)
- Timing special pentru schimbarea de canal/referință

[0]Sursă: Datasheet ATMega324a

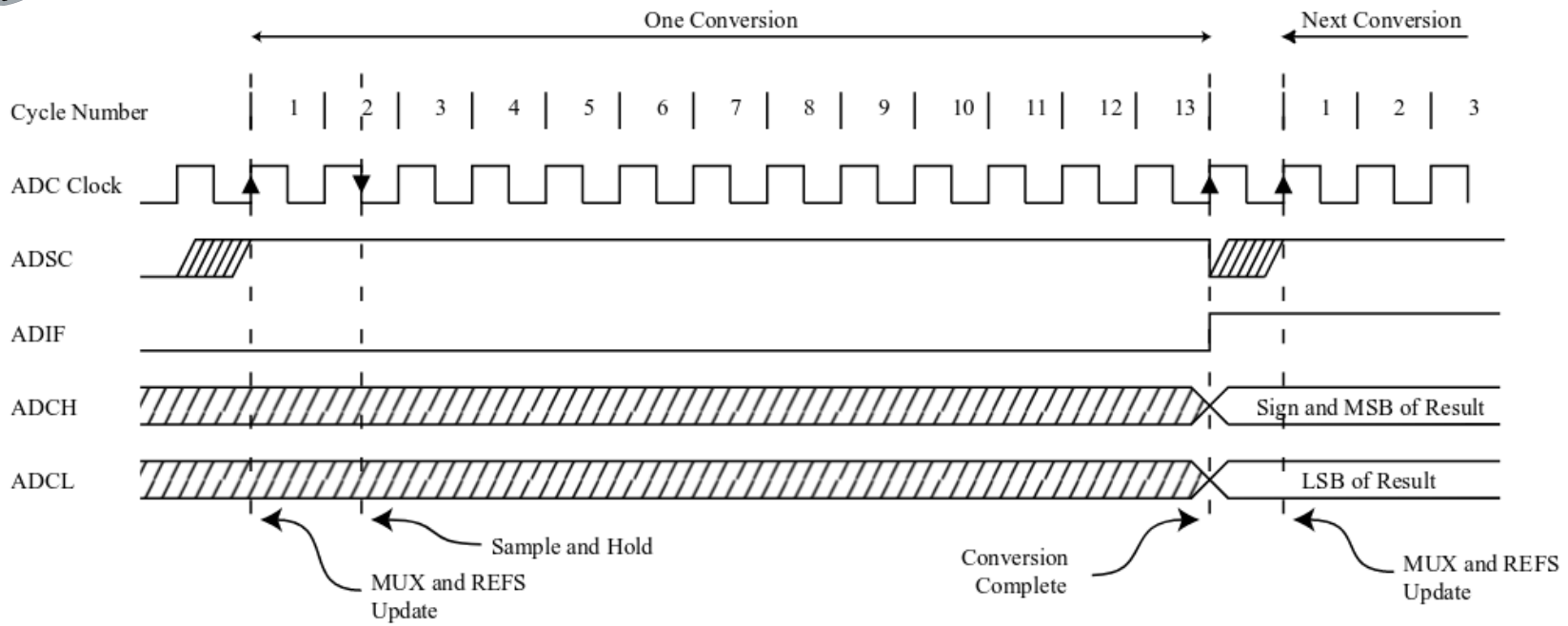
ADC - timing pentru o conversie simplă



- O conversie durează 13.x cicli ADC (prima 25)
- Timing special pentru schimbarea de canal/referință
  - Trebuie evitate schimbări de la startul conversiei până la un ciclu ADC după (maxim $8\mu s$ sau 128 cicli procesor)

[0]Sursă: Datasheet ATMega324a

ADC - timing pentru o conversie simplă

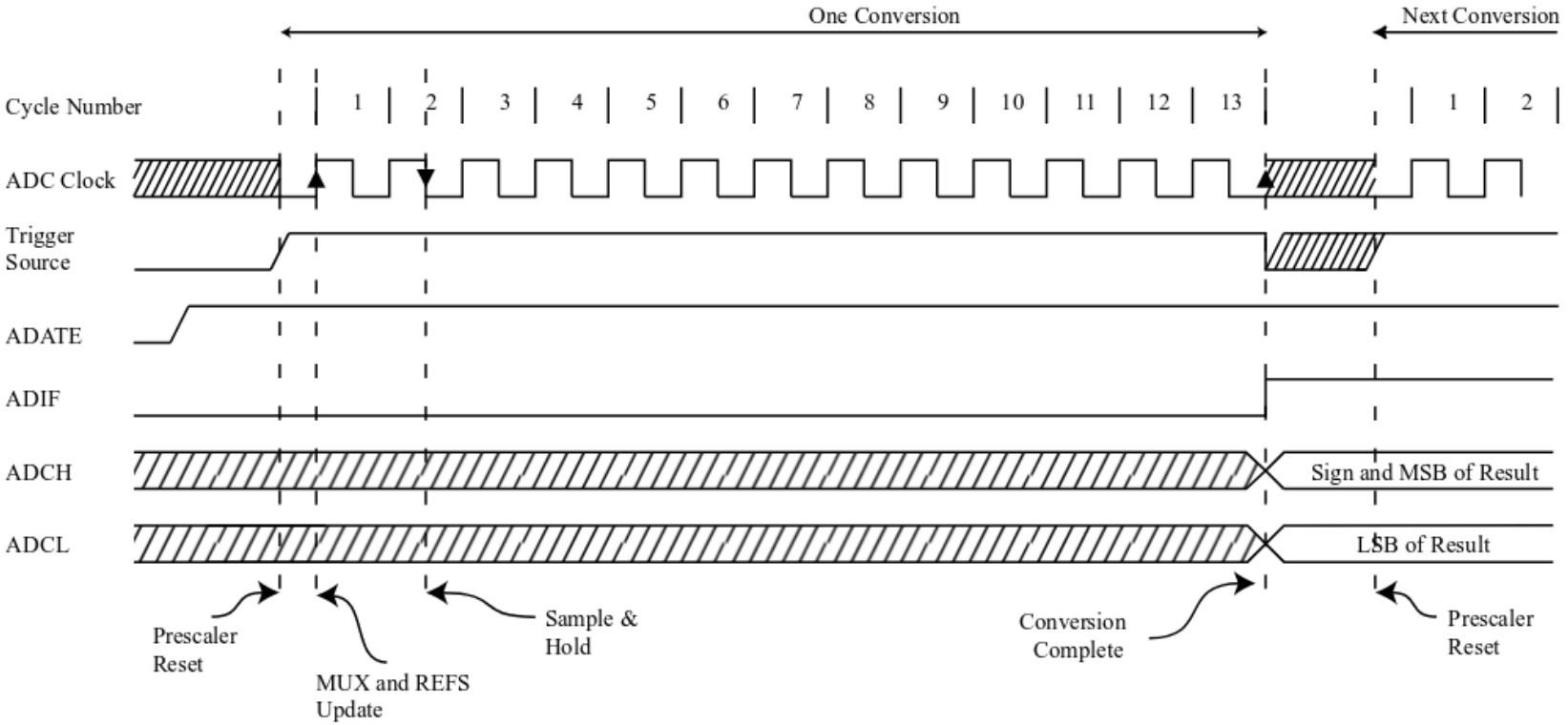


- O conversie durează 13.x cicli ADC (prima 25)
- Timing special pentru schimbarea de canal/referință
  - Trebuie evitate schimbări de la startul conversiei până la un ciclu ADC după (maxim $8\mu s$ sau 128 cicli procesor)
- După 1.5 cicli ADC se face *Sample&Hold*

[0]Sursă: Datasheet ATMega324a

- Permite pornirea automată de conversii
- Surse posibile:
  - La terminarea conversiei precedente
  - Întrerupere externă `INT0`
  - Timer0 compA sau overflow
  - Timer1 compB/overflow/input capture
  - Comparatorul analogic
- Metodă rapidă de a porni conversii

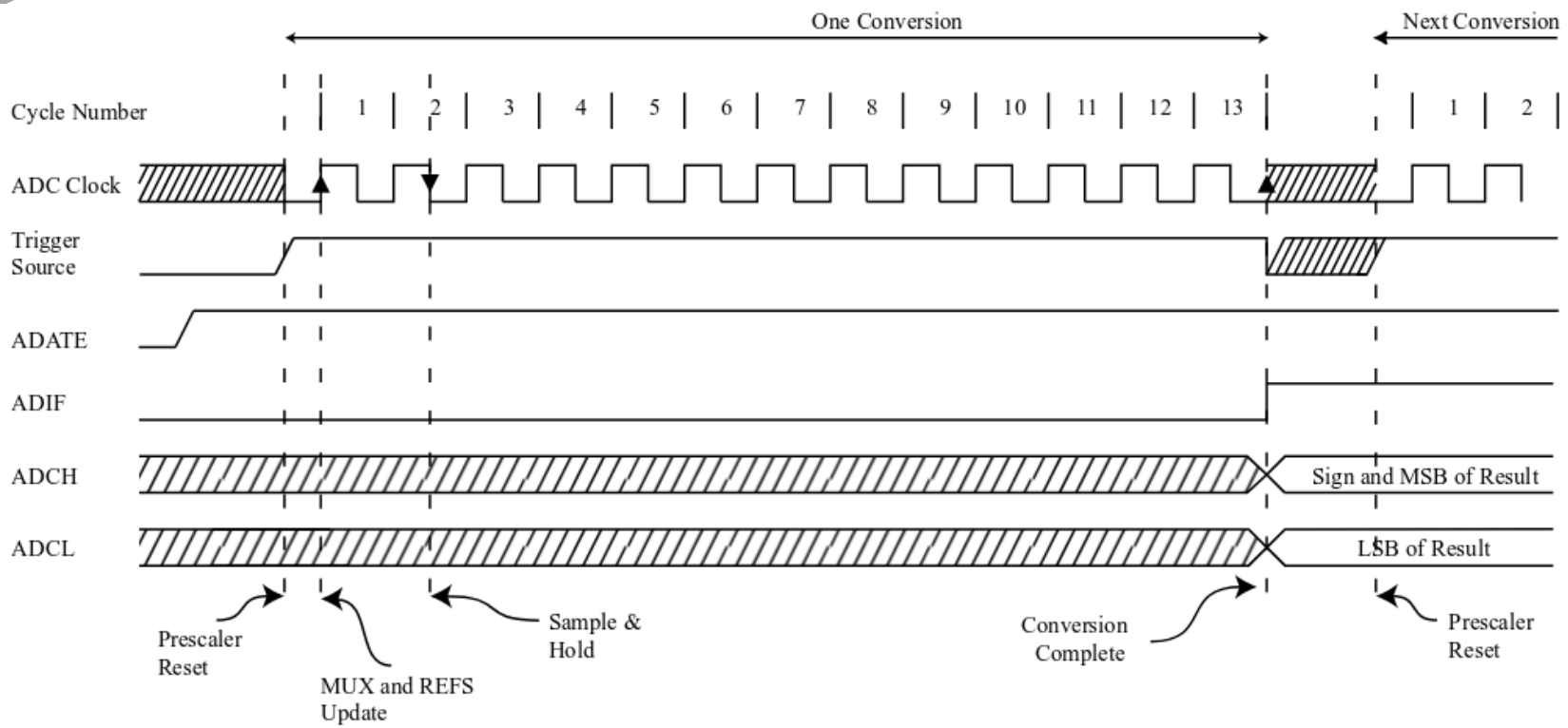
ADC - timing auto-triggering



- Durează 13.5 cicli ADC
- Mai sensibilă la schimbări de canal/referință

[0]Sursă: Datasheet ATMega324a

ADC - timing auto-triggering



- Durează 13.5 cicli ADC
- Mai sensibilă la schimbări de canal/referință
  - Schimbăm canalul în timpul unei conversii, la un ciclu ADC după pornire

[0]Sursă: Datasheet ATMega324a

ADC - exemplu auto-triggering

```
init_adc()
{

    ADMUX  = (1 << REFS0);          // initializare domeniu 0..Vcc, canalul 0
    ADCSRA = (1 << ADEN)  |         // activare ADC
             (5 << ADPS0) |         // prescaler 128
             (1 << ADATE);          // auto-triggering enabled
    ADCSRB = (5 << ADTS0);          // auto-trigger Timer 1 comp B
}
init_timer()
{
    TCCR1B = (1 << WGM12) |             // CTC, TOP la OCR1A
             (5 << CS10);               // prescaler 1024

    OCR1B = 16000000UL/1024/10 - 1; // 10Hz - 100ms
    OCR1A = OCR1B;                  // trigger-ul e pe OCR1B
}
int main()
{
  init_timer();
  init_adc();

  while(1)
  {
    // dupa primele 100ms ADC este tot timpul
    // o valoare recenta
  }
}
```

- Folosire cu mai mulți senzori
- Avem mai multe variante:
  - Pornim o conversie de fiecare dată când avem nevoie de o valoare
  - Facem conversiile pentru toți senzorii și stocăm cea mai recentă valoare

- Folosire cu mai mulți senzori
- Avem mai multe variante:
  - Pornim o conversie de fiecare dată când avem nevoie de o valoare
  - Facem conversiile pentru toți senzorii și stocăm cea mai recentă valoare
- Dacă vrem să facem toate conversiile, putem face în mai multe moduri
  - Auto-running mode: Lăsăm fiecare conversie să "cascadeze" în următoarea. Avem grijă de fiecare dată să schimbăm canalul pentru a face conversiile pentru toți senzorii
  - Timer care pornește o cascadă de conversii (o dată la X ms se pornește prima conversie, care o antrenează pe următoarea etc.)

ADC - exemplu complex

```
volatile uint8_t channel = 0;
volatile uint8_t adc_values[NRSENSORS];

ISR(TIMER1_COMPA_vect)
{
    ADCSRA |= (1 << ADSC);
}
ISR(ADC_vect)
{
    adc_values[channel] = ADC;                  // luam rezultatul precedent
    channel = (channel + 1) % NRSENSORS;        // trecem la urmatorul canal
    ADMUX = (ADMUX & 0xE0) | channel;           // schimbam canalul si punem in ADMUX
    if (channel != 0)                           // pornim urmatoarea conversie daca nu
    {                                           // este ultima
        ADCSRA |= (1 << ADSC);
    }
}
init_adc()
{
    ADMUX = (1 << REFS0);                       // referinta de tensiune AVCC
    ADCSRA = (1 << ADEN) | (1 << ADIE) | (5 << ADPS0);
}
init_timer()
{
    TCCR1B = (1 << WGM12) | (5 << CS10);        // CTC, TOP OCR1A, prescaler 1024
    TIMSK1 = (1 << OCIE1A);                     // activare intrerupere compare

    OCR1A = 16000000UL/1024/10 - 1;             // 10Hz - 100ms
}
int main()
{
    ...
}
```

ADC - exemplu complex (II)

```
volatile uint8_t channel = 0;
volatile uint8_t adc_values[NRSENSORS];

ISR(ADC_vect)
{
    adc_values[channel] = ADC;                  // luam rezultatul precedent
    channel = (channel + 1) % NRSENSORS;        // trecem la urmatorul canal
    ADMUX = (ADMUX & 0xE0) | channel;           // schimbam canalul si punem in ADMUX
    if (channel != 0)                           // pornim urmatoarea conversie daca nu
    {                                           // este ultima
        ADCSRA |= (1 << ADSC);
    }
}
init_adc()
{
    ADMUX = (1 << REFS0);                       // referinta de tensiune AVCC
    ADCSRA = (1 << ADEN) | (1 << ADATE) | (1 << ADIE) | (5 << ADPS0);
    ADCSRB = (6 << ADTS0);                      // trigger pe timer 1 overflow
}
init_timer()
{
    TCCR1A = (3 << WGM11);
    TCCR1B = (3 << WGM12) | (5 << CS10);        // Fast PWM, TOP OCR1A, prescaler 1024

    OCR1A = 16000000UL/1024/10 - 1;             // 10Hz - 100ms
}
int main()
{
    ...
}
```

- Putem converti o tensiune de pe pinii PA0..PA7
- Precizie vs. Rezoluție
- Proces lung $\rightarrow$ trebuie evitat busy-waiting
- Update-ul de configurație are perioade critice neprotejate
- Conversiile pot dura diferit în funcție de stare/configurare



Outline

ADC

EEPROM

Fuse bits

Bootloader support

- Electronically-Erasable Programmable Read-Only Memory
- Memorie non-volatilă
- Folosită sub diferite forme înainte apariției memoriei Flash
- În prezent folosită doar pentru memorie de configurare

- 1KB de memorie, adresabilă la byte
- 100000 cicli de scriere
- `EEAR` - Adresa pe 10 biți
- `EEDR` - buffer date
- `EECR` - Control Register
  - EEPM - Mode bits
  - EEMPE - Master write
  - EEPE - Write enable
  - EERE - Read enable

EEPM[1:0]	Programming Time	Operation
00	3.4ms	Erase and Write in one operation (Atomic Operation)
01	1.8ms	Erase Only
10	1.8ms	Write Only
11	-	Reserved for future use

- Operația de write durează mult

EEPM[1:0]	Programming Time	Operation
00	3.4ms	Erase and Write in one operation (Atomic Operation)
01	1.8ms	Erase Only
10	1.8ms	Write Only
11	-	Reserved for future use

- Operația de write durează mult
  - Trebuie verificat statusul `EEPE`

EEPM[1:0]	Programming Time	Operation
00	3.4ms	Erase and Write in one operation (Atomic Operation)
01	1.8ms	Erase Only
10	1.8ms	Write Only
11	-	Reserved for future use

- Operația de write durează mult
  - Trebuie verificat statusul `EEPE`
- Operația de read durează o instrucțiune

EEPM[1:0]	Programming Time	Operation
00	3.4ms	Erase and Write in one operation (Atomic Operation)
01	1.8ms	Erase Only
10	1.8ms	Write Only
11	-	Reserved for future use

- Operația de write durează mult
  - Trebuie verificat statusul `EEPE`
- Operația de read durează o instrucțiune
  - Nu trebuie verificat statusul `EERE`

EEPM[1:0]	Programming Time	Operation
00	3.4ms	Erase and Write in one operation (Atomic Operation)
01	1.8ms	Erase Only
10	1.8ms	Write Only
11	-	Reserved for future use

- Operația de write durează mult
  - Trebuie verificat statusul `EEPE`
- Operația de read durează o instrucțiune
  - Nu trebuie verificat statusul `EERE`
- Operația de read nu poate avea loc concomitent

EEPM[1:0]	Programming Time	Operation
00	3.4ms	Erase and Write in one operation (Atomic Operation)
01	1.8ms	Erase Only
10	1.8ms	Write Only
11	-	Reserved for future use

- Operația de write durează mult
  - Trebuie verificat statusul `EEPE`
- Operația de read durează o instrucțiune
  - Nu trebuie verificat statusul `EERE`
- Operația de read nu poate avea loc concomitent
  - Trebuie verificat statusul `EEPE`

EEPM[1:0]	Programming Time	Operation
00	3.4ms	Erase and Write in one operation (Atomic Operation)
01	1.8ms	Erase Only
10	1.8ms	Write Only
11	-	Reserved for future use

- Operația de write durează mult
  - Trebuie verificat statusul `EEPE`
- Operația de read durează o instrucțiune
  - Nu trebuie verificat statusul `EERE`
- Operația de read nu poate avea loc concomitent
  - Trebuie verificat statusul `EEPE`
- `EEMPE` trebuie activat cu 4 cicli înainte de `EEPE`
- `EEAR` trebuie să rămână valid

EEPM[1:0]	Programming Time	Operation
00	3.4ms	Erase and Write in one operation (Atomic Operation)
01	1.8ms	Erase Only
10	1.8ms	Write Only
11	-	Reserved for future use

- Operația de write durează mult
  - Trebuie verificat statusul `EEPE`
- Operația de read durează o instrucțiune
  - Nu trebuie verificat statusul `EERE`
- Operația de read nu poate avea loc concomitent
  - Trebuie verificat statusul `EEPE`
- `EEMPE` trebuie activat cu 4 cicli înainte de `EEPE`
- `EEAR` trebuie să rămână valid
  - Trebuie dezactivate întreruperile

- Scriere simplă

```
void EEPROM_write( unsigned int uiAddress, unsigned char ucData)
{
  /* Wait for completion of previous write */
  while (EECR & (1<<EEPE)) ;
  /* Set up address and Data Registers */
  EEAR = uiAddress;
  EEDR = ucData;
  /* Write logical one to EEMPE */
  EECR |= (1<<EEMPE);
  /* Start eeprom write by setting EEPE */
  EECR |= (1<<EEPE);
}
```

- Citire simplă

```
unsigned char EEPROM_read( unsigned int uiAddress)
{
  /* Wait for completion of previous write */
  while (EECR & (1<<EEPE)) ;
  /* Set up address register */
  EEAR = uiAddress;
  /* Start eeprom read by writing EERE */
  EECR |= (1<<EERE);
  /* Return data from Data Register */
  return EEDR;
}
```

[0]Sursă: Datasheet ATMega324a

- Singura memorie non-volatilă ușor accesibilă
- Constrângerile memoriei $\rightarrow$ folosire anevoioasă
- Sensibilitate mare la întreruperi în faza de comandă



Outline

ADC

EEPROM

Fuse bits

Bootloader support



Ce sunt fuse bits?

- Biți de configurare de bază
- Accesibili (mai ales) din exterior

- `JTAGEN` - enable programare pe JTAG
- `OCDEN` - enable debug pe JTAG
- `SPIEN` - enable programare prin SPI
- `WDTON` - watchdog timer always on
- `EESAVE` - EEPROM se păstrează chiar și la erase
- `BOOTSZ` - dimensiune bootloader
- `BOOTRST` - funcționare cu bootloader

- `CKDIV8` - împarte ceasul la 8
- `CKOUT` - ceasul vizibil pe `PB0`
- `SUT` - timp de start-up
- `CKSEL` - sursă ceas
  - Cel mai periculos câmp
  - Internal oscillator
  - External clock
  - Low Power crystal Oscillator
  - Low Frequency crystal oscillator
  - ...



Fuse byte extended

- `BODLEVEL` - Brown-out detection
- Nivele de tensiune la 1.8V, 2.7V, 4.3V

- CKDIV8
  - CLKPS este prescaler-ul ceasului [1..256]

```
CLKPR = (1 << CLKPCE); // change enable
CLKPR = 0;             // CLKPS3..0 = 0 - prescaler 1
```

- JTAGEN
  - Bit-ul JTD din MCUCR

```
uint8_t temp = MCUCR;
MCUCR = temp | (1 << JTD);
MCUCR = temp | (1 << JTD); // in maxim 4 cicli
```

- BOOTRST
  - Bit-ul IVSEL din MCUCR

```
uint8_t temp = MCUCR;
MCUCR = temp | (1 << IVCE);    // change enable
MCUCR = temp & ~(1 << IVSEL); // in maxim 4 cicli
```



Concluzii fuse bits

- Configurații importante
- Set & forget



Outline

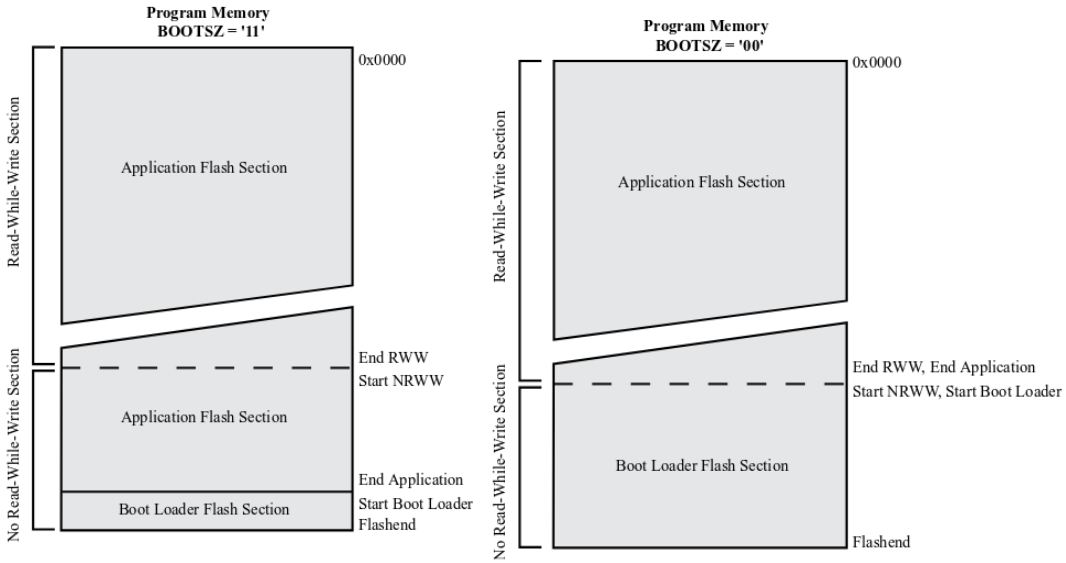
ADC

EEPROM

Fuse bits

Bootloader support

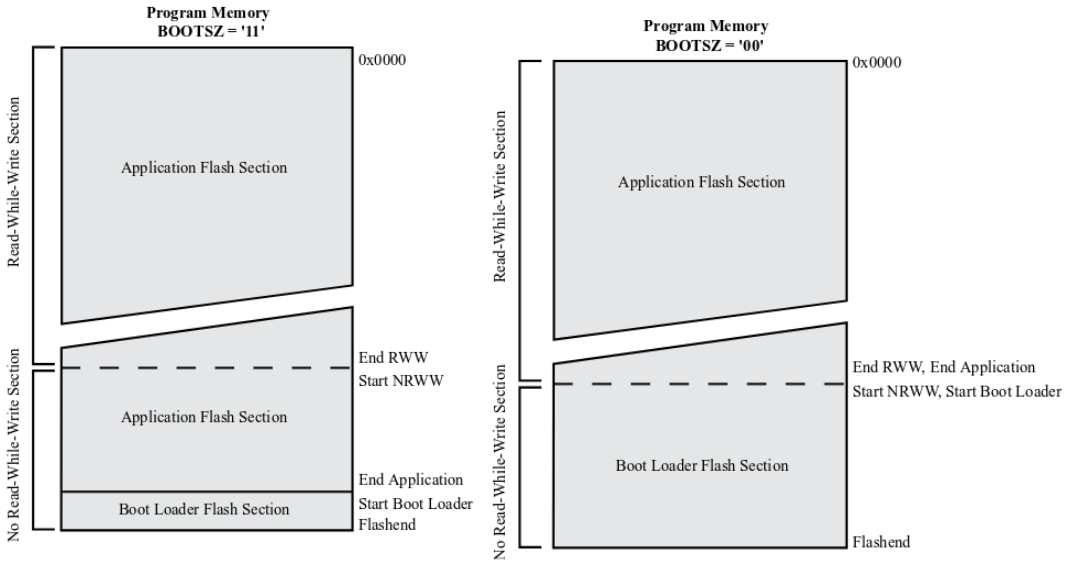
- Constrângeri specifice
  - Număr limitat de scrieri
  - Scrierile pot schimba doar biți de 1 în 0
  - Erase duce biții la 1
  - Erase se face doar pe pagini - 64 cuvinte - 128 bytes
  - Erase durează între $3.7ms$ și $4.5ms$
- Pentru bootloader
  - Memoria Flash pe care o scrie e aceeași din care execută
  - Scrierea e mult mai lentă decât ceea ce poate primi



- Este permisă citirea în zona bootloader-ului când se scrie/șterge în alte zone

[0]Sursă: Datasheet ATMega324a

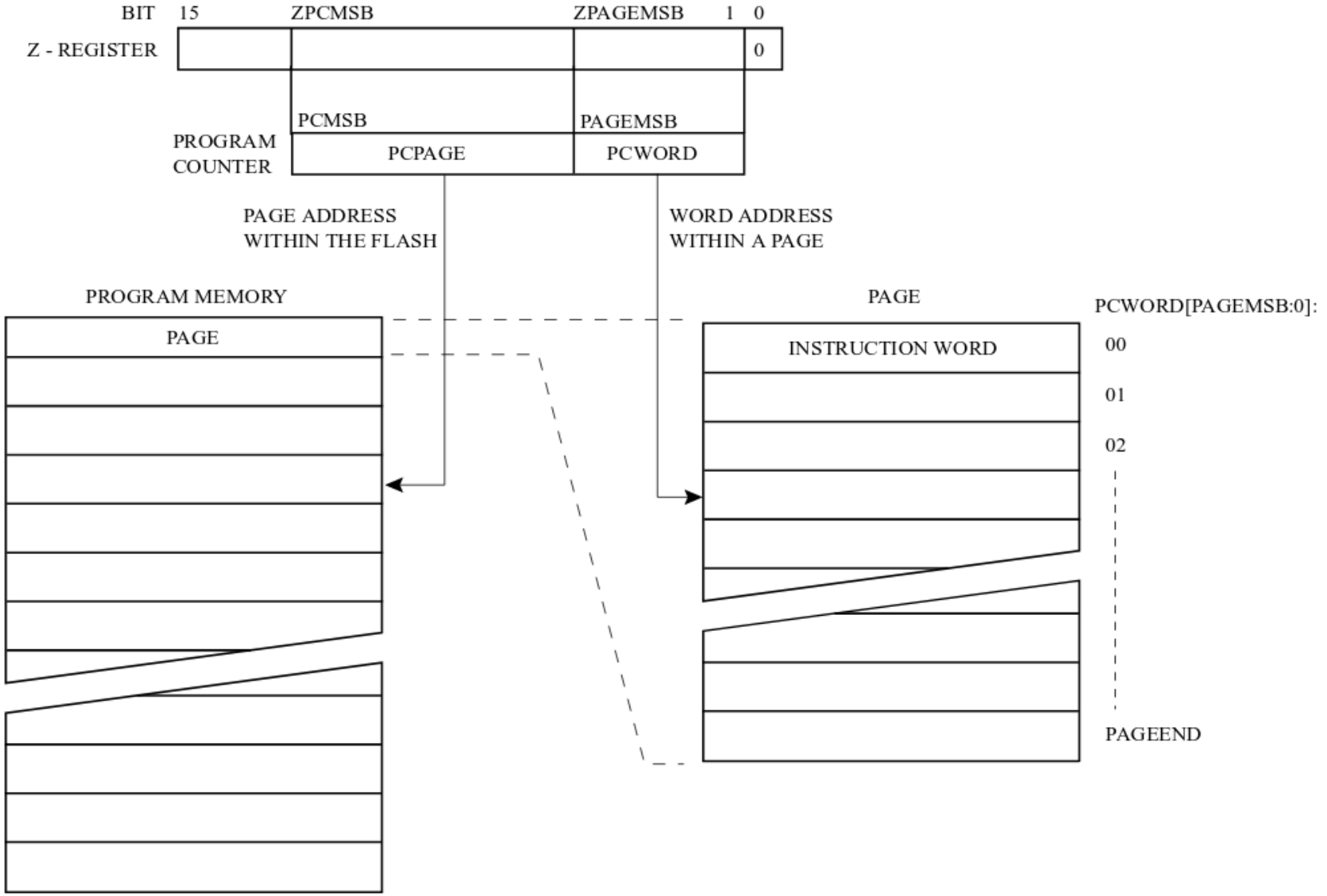
ATMega324a - Read-While-Write



- Este permisă citirea în zona bootloader-ului când se scrie/șterge în alte zone
- Uneori însă bootloader-ul nu acoperă întreaga zonă

[0]Sursă: Datasheet ATMega324a

Adresare



[0]Sursă: Datasheet ATMega324a

- Scriere RAM → page buffer
  - Câte un word (2 bytes)
  - `Buffer[PCWORD] = R1:R0`

- Scriere RAM → page buffer
  - Câte un word (2 bytes)
  - `Buffer[PCWORD] = R1:R0`
- Ștergerea unei pagini
  - Câte o pagină (256 bytes)
  - Adresa este `PCPAGE`

- Scriere RAM → page buffer
  - Câte un word (2 bytes)
  - `Buffer[PCWORD] = R1:R0`
- Ștergerea unei pagini
  - Câte o pagină (256 bytes)
  - Adresa este `PCPAGE`
- Scrierea unei pagini
  - Câte o pagină (256 bytes)
  - `Flash[PCPAGE] = Buffer`

- Scriere RAM → page buffer
  - Câte un word (2 bytes)
  - `Buffer[PCWORD] = R1:R0`
- Ștergerea unei pagini
  - Câte o pagină (256 bytes)
  - Adresa este `PCPAGE`
- Scrierea unei pagini
  - Câte o pagină (256 bytes)
  - `Flash[PCPAGE] = Buffer`
- Fiecare operație presupune

- Scriere RAM → page buffer
  - Câte un word (2 bytes)
  - `Buffer[PCWORD] = R1:R0`
- Ștergerea unei pagini
  - Câte o pagină (256 bytes)
  - Adresa este `PCPAGE`
- Scrierea unei pagini
  - Câte o pagină (256 bytes)
  - `Flash[PCPAGE] = Buffer`
- Fiecare operație presupune
  - Pregătire parametri în `Z (R31:R30)`, `R1`, `R0`



Trei tipuri de operații

- Scriere RAM → page buffer
  - Câte un word (2 bytes)
  - `Buffer[PCWORD] = R1:R0`
- Ștergerea unei pagini
  - Câte o pagină (256 bytes)
  - Adresa este `PCPAGE`
- Scrierea unei pagini
  - Câte o pagină (256 bytes)
  - `Flash[PCPAGE] = Buffer`
- Fiecare operație presupune
  - Pregătire parametri în `Z (R31:R30)`, `R1`, `R0`
  - Selectare operație în `SPMCSR`



Trei tipuri de operații

- Scriere RAM → page buffer
  - Câte un word (2 bytes)
  - `Buffer[PCWORD] = R1:R0`
- Ștergerea unei pagini
  - Câte o pagină (256 bytes)
  - Adresa este `PCPAGE`
- Scrierea unei pagini
  - Câte o pagină (256 bytes)
  - `Flash[PCPAGE] = Buffer`
- Fiecare operație presupune
  - Pregătire parametri în `Z (R31:R30)`, `R1`, `R0`
  - Selectare operație în `SPMCSR`
  - Instrucțiune `SPM`



Pseudocod bootloader

```
ISR(USART0_RX_vect)
{
  // Add to buf, buf_size
}
int main()
{
  init_usart();

  while(1)
  {
    while(buf_size < PAGESIZE * 2);

    bl_fill_pagebuffer(buf);
    bl_erasepage(current_page);
    bl_writepage(current_page++);

    // presupunem ca exista un flow control, pe seriala
    // se transmite cate o singura pagina si
    // se asteapta un ack pentru a trimite urmatoare pagina
    // si se transmit paginile in ordine, fara a depasi capacitatea
    send_ack();
  }
}
```

- `spm` nu poate fi executată decât din zona de bootloader
- Lock bits
  - Asemănători cu fuse bits
  - 2 biți pentru Application section
  - 2 biți pentru Bootloader section
  - Pot preveni citiri/scrieri de memorie de program
  - Previn suprascrieri accidentale ale bootloader-ului
  - Previn accesul la codul precedent

- Scrierea pe flash
  - Erase pe blocuri
  - Write cu pregătirea unui buffer
- Read-While-Write ne ajută
  - Putem executa cod de bootloader cât timp el șterge un bloc
  - Nu este însă valabil pe toată memoria
- Instrucțiunile de spm nu pot fi executate decât din bootloader